have any effect whatsoever. If no users are left after we've deleted the user, we'll roll the transaction back, restoring the user we just deleted.

To do this, we'll use an Active Record hook method. We've already seen one of these: the `validate` hook is called by Active Record to validate an object's state. It turns out that Active Record defines sixteen or so hook methods, each called at a particular point in an object's life cycle. We'll use the `after_destroy` hook, which is called after the SQL `delete` is executed. If a method by this name is publicly visible, it'll conveniently be called in the same transaction as the `delete`—so if it raises an exception, the transaction will be rolled back. The hook method looks like this:

rails7/depot_t/app/models/user.rb

```
after_destroy :ensure_an_admin_remains

class Error < StandardError
end

private
  def ensure_an_admin_remains
    if User.count.zero?
      raise Error.new "Can't delete last user"
    end
  end
```

The key concept is the use of an exception to indicate an error when the user is deleted. This exception serves two purposes. First, because it's raised inside a transaction, it causes an automatic rollback. By raising the exception if the `users` table is empty after the deletion, we undo the delete and restore that last user.

Second, the exception signals the error back to the controller, where we use a `rescue_from` block to handle it and report the error to the user in the notice. If you want only to abort the transaction but not otherwise signal an exception, raise an `ActiveRecord::Rollback` exception instead, because this is the only exception that won't be passed on by `ActiveRecord::Base.transaction`:

rails7/depot_t/app/controllers/users_controller.rb